

SQL

Revision Sheet – Quick Reference

What is SQL?

SQL = Structured Query Language. It's how you talk to databases.

You use it to SELECT, filter, group, join, and aggregate tabular data.

Syntax is almost identical across SQLite, MySQL, PostgreSQL, BigQuery.

Learn it once — works everywhere. For practice: SQLite (zero setup, built into Python).

SQL vs Pandas:

- WHERE → `df[df['col'] > x]`
- ORDER BY → `df.sort_values()`
- GROUP BY → `df.groupby()`
- JOIN → `pd.merge()`
- Window func → no direct pandas equivalent (`rank`, `lead`, `lag`)

```
-- standard query order:
SELECT  columns
FROM    table
JOIN    other_table ON condition
WHERE   row_filter
GROUP BY column
HAVING  group_filter
ORDER BY column DESC
LIMIT  n;
```

01 SELECT + WHERE + ORDER BY + LIMIT

SELECT fetches columns. WHERE filters rows. ORDER BY sorts. LIMIT caps result size.

WHERE operators: = != > < >= <= AND OR. DISTINCT removes duplicate values.

```
SELECT * FROM students;           -- all rows, all cols
SELECT name, marks FROM students; -- specific cols

SELECT * FROM students WHERE marks > 80; -- filter rows
SELECT * FROM students WHERE city = 'Delhi' AND marks > 80;

SELECT * FROM students ORDER BY marks DESC; -- sort high→low
SELECT * FROM students ORDER BY marks DESC LIMIT 3; -- top 3

SELECT DISTINCT city FROM students; -- unique cities only
-- NOTE: DISTINCT applies to the whole row, not one column
```

02 Aggregate Functions

COUNT(*) counts all rows. COUNT(col) skips NULLs in that column — common bug source.

```
SELECT COUNT(*), SUM(marks), AVG(marks), MIN(marks), MAX(marks)
FROM students;
-- COUNT(*): 6 | AVG: 85.00 | MIN: 72 | MAX: 95
```

03 GROUP BY + HAVING

GROUP BY groups rows and applies aggregate per group. Like `df.groupby()` in pandas.

Rule: every col in SELECT must be in GROUP BY or inside an aggregate function.

WHERE filters rows before grouping. HAVING filters groups after grouping.

```
SELECT city, COUNT(*), ROUND(AVG(marks), 2)
FROM students
GROUP BY city;

-- filter groups: only cities with more than 1 student
SELECT city, COUNT(*) FROM students
GROUP BY city
HAVING COUNT(*) > 1;

-- Wrong: WHERE COUNT(*) > 1  ← can't use aggregate in WHERE
-- Correct: HAVING COUNT(*) > 1
```

04 LIKE + IN + BETWEEN

LIKE: pattern match. % = any chars, _ = exactly one char.

IN: match multiple values (cleaner than OR). BETWEEN: range, both ends inclusive.

```
SELECT * FROM students WHERE name LIKE 'A%';  -- starts with A
SELECT * FROM students WHERE name LIKE '%an%'; -- contains 'an'
SELECT * FROM students WHERE name LIKE '_r%';  -- 2nd letter is r

SELECT * FROM students WHERE city IN ('Delhi', 'Mumbai');
-- same as: WHERE city='Delhi' OR city='Mumbai'

SELECT * FROM students WHERE marks BETWEEN 80 AND 90;
-- includes 80 and 90 (unlike Python range which excludes end)

-- combine: WHERE marks BETWEEN 85 AND 95 AND city IN ('Delhi','Bangalore')
```

05 JOINS

Combine two tables using a common column. Like `pd.merge()` in pandas.

INNER: only matches. LEFT: all of left. RIGHT: all of right. FULL OUTER: everything.

```
-- INNER JOIN: only rows that match on both sides
SELECT students.name, courses.course
FROM students
INNER JOIN courses ON students.id = courses.student_id;
-- Sneha(4) and Ananya(6) dropped – no course. student_id=7 dropped – no student.
```

```
-- LEFT JOIN: all students + course if exists, NULL if not
SELECT students.name, courses.course
FROM students
LEFT JOIN courses ON students.id = courses.student_id;
-- Sneha → NULL, Ananya → NULL. student_id=7 still excluded.
```

```
-- RIGHT JOIN: all courses + student if exists, NULL if not
-- student_id=7 appears with NULL name

-- FULL OUTER JOIN: everything from both sides
-- 7 rows: all 6 students + student_id=7 from courses
-- FULL OUTER: table order doesn't matter (unlike LEFT/RIGHT)
```

INNER	only matching rows
LEFT	all of left + NaN where no right match
RIGHT	all of right + NaN where no left match
FULL OUTER	everything from both, NaN where missing

06 Subquery

A query inside another query. Inner query runs first, outer query uses its result.

Subquery in FROM creates a temp table — must give it an alias (AS name) or SQL errors.

Common pattern: window function creates a col that doesn't exist yet → wrap in subquery to filter it.

```
-- instead of two queries, nest one inside:
SELECT * FROM students
WHERE marks > (SELECT AVG(marks) FROM students);

-- subquery in FROM (mandatory for filtering window func results):
SELECT * FROM (
    SELECT gender, math_score,
    RANK() OVER (PARTITION BY gender ORDER BY math_score DESC) AS rnk
    FROM student_performance
) AS ranked          -- alias is mandatory
WHERE rnk = 1;
-- SQL order: FROM→WHERE→SELECT, so window cols don't exist at WHERE stage
```

07 Window Functions

OVER() = window function. Keeps all rows but adds group calculations beside each row.

PARTITION BY divides into groups (rows don't collapse). ORDER BY sets order within group.

```
-- ROW_NUMBER: unique number per row, no ties
ROW_NUMBER() OVER (PARTITION BY gender ORDER BY math_score DESC)
-- 1, 2, 3, 4 (even if scores are equal)

-- RANK: ties get same rank, next rank skips
RANK() OVER (...) -- 1, 1, 1, 4 (skips 2 and 3)

-- DENSE_RANK: ties same rank, NO gaps
DENSE_RANK() OVER (...) -- 1, 1, 1, 2 (no skip)

-- LEAD: value from next row | LAG: value from previous row
LEAD(math_score) OVER (PARTITION BY gender ORDER BY math_score DESC)
LAG(math_score) OVER (PARTITION BY gender ORDER BY math_score DESC)
-- first row LAG = NULL, last row LEAD = NULL
-- use case: compare this month vs previous/next month
```